

УНИВЕРЗИТЕТ У НИШУ
ЕЛЕКТРОНСКИ ФАКУЛТЕТ



SEMINARSKI RAD I
Selekcija instanci u mašinskom učenju

Studijski program: Veštačka inteligencija i mašinsko učenje
Predmet: Prikupljanje i predobrada podataka za mašinsko učenje

Mentor:
prof. dr Aleksandar Stanimirović

Student:
Aleksa Cekić, br. ind. 2117

Niš, 2026.

Table of Contents

1. UVOD	4
2. PROBLEM SELEKCIJE INSTANCI	5
Prisustvo šuma u podacima	5
Nebalansiranost klasa	5
2.1 Formalna definicija problema	6
Matematička formulacija	6
Dodatna ograničenja	7
NP-težina problema	7
2.2 Tipovi selekcije instanci	8
Kondenzacija (Condensation)	8
Uređivanje (Edition)	8
Hibridni pristupi	9
3. ALGORITMI ZA SELEKCIJU INSTANCI	10
3.1 Metode kondenzacije	10
Condensed Nearest Neighbor (CNN)	10
Karakteristike CNN algoritma	11
Reduced Nearest Neighbor (RNN)	11
Minimal Subset Selection (MSS)	12
3.2 Metode uređivanja	13
Edited Nearest Neighbor (ENN)	14
Uticaj parametra k	14
Prednosti i ograničenja ENN-a	15
Repeated ENN (RENN)	15
All K-NN	16
Tomek Links	16
3.3 Hibridne metode	17
DROP algoritmi	17
DROP3 algoritam	17
Performanse DROP3	18
Iterative Case Filtering (ICF)	19
3.4 Evolucione metode	19
Genetski algoritmi (GA)	20
Particle Swarm Optimization (PSO)	21
4. EVALUACIJA	21
4.1 Metrike evaluacije	21
4.2 Oblasti primene	22
4.3 Praktične preporuke	22
5. ZAKLJUČAK	24
6. PRAKTIČAN DEO	25

6.1 Opis skupa podataka	25
6.2 Baseline i primenjene metode	26
6.3 Rezultati	28
6.4 Diskusija	30
7. LITERATURA	31

1. UVOD

Selekcija instanci je jedan od centralnih problema u mašinskom učenju koji se javlja čim trening skup postane dovoljno veliki da počne da stvara probleme.

Redundantni primeri usporavaju treniranje, pogrešno označene instance kvare granicu odlučivanja, a odstupajuće vrednosti koje ne reprezentuju nijednu klasu mogu da nauče model pogrešnim obrascima. Ovo nije samo tehničko usporavanje, već pojava koja direktno utiče na tačnost klasifikacije, memorijske zahteve i vreme predikcije.

Algoritmi zasnovani na instancama, poput k-Nearest Neighbors, posebno su osetljivi na ovaj problem. Za razliku od većine algoritama koji grade eksplicitan model tokom treniranja, k-NN čuva ceo trening skup u memoriji i koristi ga direktno pri svakoj predikciji. Za skup od 10 miliona instanci to znači da klasifikacija jednog novog primera zahteva 10 miliona izračunavanja rastojanja, što u sistemima koji zahtevaju odgovor u milisekundama postaje praktično neupotrebljivo.

Uzroci su raznovrsni. Količina podataka u savremenim aplikacijama raste eksponencijalno, realni podaci gotovo uvek sadrže određen procenat šuma u vidu pogrešno označenih instanci ili grešaka senzora, a veliki deo trening skupa čine primeri koji se nalaze duboko unutar svoje klase i ne doprinose ničemu korisnom za definisanje granica između klasa. Njihovo prisustvo samo usporava sistem bez ikakvog doprinosa tačnosti.

Istraživanja pokazuju da već 5-10% šumnih podataka može značajno da degradira performanse k-NN klasifikatora, dok uklanjanje redundantnih instanci može ubrzati predikciju bez pada tačnosti, ponekad čak i poboljšati je jer se uklanjaju instance koje su model samo zbunjivale.

Tokom decenija razvijen je čitav niz metoda za rešavanje ovog problema. Neke agresivno smanjuju veličinu skupa zadržavajući samo granične instance između klasa, druge se fokusiraju isključivo na čišćenje podataka uklanjanjem šuma, treće kombinuju oba pristupa, a postoje i evolucioni pristupi koji problem tretiraju kao zadatak globalne optimizacije. Ovaj rad sistematski prolazi kroz sve četiri kategorije, teorijske osnove, prednosti, ograničenja i preporuke za primenu.

2. PROBLEM SELEKCIJE INSTANCI

Selekcija instanci predstavlja proces odabira reprezentativnog podskupa trening podataka koji održava ili poboljšava performanse modela uz značajno smanjenje računске složenosti. Instance-based algoritmi poput k-Nearest Neighbors (k-NN) čuvaju ceo trening skup u memoriji i koriste ga direktno tokom klasifikacije, što dovodi do linearne vremenske složenosti $O(n)$ po predikciji. Za skupove sa milionima instanci, ovo postaje praktično neupotrebljivo u aplikacijama koje zahtevaju odgovore u realnom vremenu.

Nekoliko ključnih faktora motiviše primenu selekcije instanci. Prvo, računska efikasnost je kritična jer k-NN sa 10 miliona instanci zahteva 10 miliona izračunavanja rastojanja za svaku novu predikciju. Studije pokazuju da za k-NN sa 10 miliona instanci u d-dimenzionalnom prostoru, jedna predikcija zahteva $O(n \cdot d)$ operacija izračunavanja rastojanja, što može trajati nekoliko sekundi na standardnom procesoru, dok aplikacije poput detekcija prevara u realnom vremenu zahtevaju odgovore u milisekundama. Drugo, memorijski zahtevi rastu linearno sa veličinom trening skupa, što može biti prohibitivno za embedded sisteme, mobilne aplikacije ili edge computing uređaje sa ograničenom RAM memorijom.

Prisustvo šuma u podacima

Šum u podacima manifestuje se kroz nekoliko oblika. Pogrešno označene instance nastaju usled ljudske greške tokom procesa etiketiranja, nesavršenih automatskih sistema za prikupljanje podataka, ili promena u definiciji klasa tokom vremena. Na primer, u medicinskim skupovima podataka, različiti lekari mogu imati različite interpretacije iste dijagnoze, što dovodi do nedoslednih oznaka. Outlieri predstavljaju atipične primere koji se nalaze daleko od normalne distribucije svoje klase i mogu nastati usled grešaka u merenju, korupcije podataka tokom prenosa, ili retkih neuobičajenih slučajeva koji ne reprezentuju opštu populaciju.

Redundantne instance su primeri koji se nalaze u unutrašnjosti oblasti svoje klase, daleko od granica odlučivanja, i ne doprinose dodatnim informacijama o strukturi podataka. Prisustvo ovih tipova šuma može dovesti do dva suprotna problema: overfitting kada model uči specifične karakteristike šumnih instanci umesto pravih obrazaca u podacima, i underfitting kada veliki broj redundantnih instanci razblažuje signal od informativnih graničnih instanci. Empirijske studije na benchmark skupovima podataka pokazuju da uklanjanje samo 10-20% najšumnijih instanci može dovesti do poboljšanja tačnosti klasifikacije za 2-5 procentnih poena, istovremeno smanjujući vreme treniranja i predikcije za faktor 5-10.

Nebalansiranost klasa

Nebalansiranost klasa predstavlja dodatni izazov koji motiviše primenu selekcije instanci. U mnogim realnim aplikacijama, kao što su medicinska dijagnostika gde retke bolesti mogu činiti manje od 1% slučajeva, detekcija prevara gde su samo 0.1-0.5% transakcija lažne, ili detekcija napada u računarskim mrežama gde normalan saobraćaj

dominira sa preko 99% paketa, jedna klasa drastično dominira brojnošću u odnosu na drugu.

Standard k-NN algoritam u ovakvim scenarijima pokazuje tendenciju da favorizuje većinsku klasu jer većina najbližih suseda nove instance verovatno pripada toj klasi. Na primer, u skupu sa 1000 legitimnih transakcija i 10 lažnih, k-NN sa $k=5$ će praktično uvek klasifikovati novu instancu kao legitimnu jer je verovatnoća da se svih 5 najbližih suseda nalazi u većinskoj klasi veoma visoka.

Selekcija instanci može adresirati ovaj problem kroz targetirano uklanjanje redundantnih instanci iz većinske klase, efektivno balansujući distribuciju bez potrebe za veštačkom generacijom novih primeraka kroz tehnike kao što je SMOTE. Dodatno, u visokodimenzionim prostorima atributa dolazi do pojave poznate kao “curse of dimensionality” gde rastojanja između tačaka postaju manje diskriminativne, a koncept blizine gubi značenje jer sve tačke postaju približno jednako udaljene jedna od druge.

2.1 Formalna definicija problema

Selekcija instanci može se formalno definisati kao problem optimizacije gde je cilj naći podskup trening podataka koji optimizuje određenu funkciju cilja. Neka je dat trening skup $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, gde je $x_i \in \mathbb{R}^d$ vektor atributa d-dimenzionalnog prostora obeležja, a $y_i \in \{1, 2, \dots, c\}$ odgovarajuća klasna oznaka iz skupa od c klasa.

Cilj selekcije instanci je pronaći podskup $S \subseteq T$ takav da zadovoljava dva glavna uslova:

- $|S| \ll |T|$ što znači da je kardinalitet selektovanog podskupa značajno manji od kardinaliteta originalnog skupa
- $\text{Acc}(S) \approx \text{Acc}(T)$ što znači da tačnost klasifikatora treniranog na podskupu S treba da bude približno jednaka ili veća od tačnosti klasifikatora treniranog na celokupnom skupu T

Ova formulacija predstavlja multi-objective optimization problem sa konfliktnim ciljevima jer povećanje stope redukcije često dovodi do smanjenja tačnosti modela, i obrnuto, zadržavanje veće tačnosti zahteva uključivanje više instanci u selektovani podskup.

Matematička formulacija

Optimizacioni problem može se izraziti kao maksimizacija kompozitne funkcije cilja:

$$\text{maximize } f(S) = \alpha \cdot R(S) + (1-\alpha) \cdot A(S)$$

$$\text{subject to } S \subseteq T$$

gde je:

- $R(S) = 1 - |S|/|T|$ stopa redukcije izražena kao procenat smanjenja veličine skupa

- $A(S) = \text{Accuracy}(h_{\square}) / \text{Accuracy}(h_{\square})$ relativna tačnost gde je $h_{\square}$ klasifikator treniran na S a $h_{\square}$ na T
- $\alpha \in [0,1]$ parametar koji balansira relativnu važnost redukcije naspram tačnosti

Izbor vrednosti parametra α zavisi od prioriteta aplikacije: u scenarijima gde je brzina kritična kao što su real-time sistemi za detekciju upada ili high-frequency trading, α može biti blizu 1 favorizujući agresivnu redukciju čak i uz izvestan pad tačnosti, dok u aplikacijama gde je tačnost presudna kao što je medicinska dijagnostika ili kontrola kvaliteta u kritičnim industrijskim procesima, α treba biti bliže 0 jer čak i mali pad tačnosti može imati ozbiljne posledice.

Dodatna ograničenja

Neki pristupi uvode dodatna ograničenja poput:

- Minimalnog praga tačnosti: $A(S) \geq \theta$ gde se specifikuje da tačnost ne sme pasti ispod zadatog nivoa
- Maksimalnog dozvoljenog pada tačnosti: $|A(S) - 1| \leq \varepsilon$ što garantuje da relativni pad tačnosti ne prelazi epsilon vrednost, obično postavljenu na 1-5%

Dodatna dimenzija problema je tip funkcije cilja koja se optimizuje i strategija pretrage kroz prostor mogućih podskupova. Metode omotača direktno koriste performanse klasifikatora kao funkciju cilja, što rezultuje u podskupovima optimizovanim za konkretan algoritam ali sa većom računskom složenošću jer svaka evaluacija kandidatnog podskupa zahteva treniranje i testiranje modela kroz cross-validation. Filter metode koriste heuristike zasnovane na karakteristikama podataka kao što su konzistentnost, udaljenosti između instanci, gustina distribucije, ili entropija, što je računski efikasnije sa složenošću tipično $O(n^2)$ ili $O(n^2 \cdot d)$ ali može rezultovati u suboptimalnim rešenjima jer ne uzima u obzir specifičnosti klasifikatora koji će na kraju koristiti selektovane podatke. Hibridne metode kombinuju oba pristupa, prvo koristeći filter za brzu predikciju kandidata sa vremenom $O(n^2)$, a zatim omotač za finu optimizaciju na redukovanom skupu sa $O(m^2)$ gde je $m \ll n$, postižući dobar balans između kvaliteta i efikasnosti.

NP-težina problema

Važno je napomenuti da je problem nalaženja minimalnog konzistentnog podskupa (Minimal Consistent Set problem), gde se traži najmanji podskup S koji pravilno klasifikuje sve instance iz T , formalno NP-hard problem. To znači da ne postoji efikasan algoritam koji garantuje nalaženje optimalnog rešenja u polinomijalnom vremenu za opšti slučaj. Zbog toga se u praksi koriste heuristički i aproksimativni pristupi kao što su pohlepni algoritmi, genetski algoritmi, ili lokalna pretraga. Ovi pristupi ne garantuju nalaženje globalnog optimuma, ali u praksi često nalaze rešenja koja su blizu optimuma ili čak optimalna za mnoge skupove podataka.

2.2 Tipovi selekcije instanci

Instance Selection algoritmi mogu imati različite primarne ciljeve u zavisnosti od prirode podataka i zahteva aplikacije, što dovodi do nekoliko osnovnih kategorija metoda.

Kondenzacija (Condensation)

Kondenzacija predstavlja pristup gde je primarni cilj maksimalna redukcija broja instanci uz održavanje ili minimalan pad tačnosti klasifikacije. Metode kondenzacije fokusiraju se na identifikaciju i zadržavanje kritičnih instanci koje definišu granice odlučivanja između klasa, pri čemu se ove instance nazivaju boundary instances (granične instance), dok se uklanjaju unutrašnje instance (internal instances) odnosno primeri koji se nalaze duboko unutar oblasti svoje klase i ne doprinose definisanju granica između klasa.

Intuicija je da za k-NN klasifikaciju, internal instances su redundantne jer najbliži susedi nove instance u unutrašnjosti klase će svakako biti iz iste klase bez obzira da li je konkretna internal instance prisutna ili ne. Tipičan rezultat kondenzacije je drastična redukcija od 70-95% originalnog skupa, pri čemu se tačnost zadržava na nivou 95-100% originalne tačnosti, što predstavlja izuzetno povoljan kompromis (trade-off).

Algoritmi kao što su Condensed Nearest Neighbor (CNN) razvijen od strane Peter E. Hart-a 1968. godine i Reduced Nearest Neighbor (RNN) od Geoffrey W. Gates-a 1972. godine pripadaju ovoj kategoriji. Kondenzacija je posebno korisna u scenarijima gde su memorijski i vremenski resursi limitirani, kao što su embedded sistemi u automobilima ili medicinskim uređajima, mobilne aplikacije koje rade na uređajima sa ograničenom RAM memorijom i procesorskom snagom, ili cloud aplikacije gde se naplaćuje po potrošnji resursa i gde redukcija veličine modela direktno utiče na operativne troškove.

Uređivanje (Edition)

Uređivanje (Edition) predstavlja pristup gde je primarni cilj poboljšanje kvaliteta podataka kroz uklanjanje šumnih i atipičnih instanci, pri čemu redukcija veličine nije primarna motivacija već posledica čišćenja podataka. Edition metode identifikuju i uklanjaju instance koje su verovatno pogrešno označene jer njihovi najbliži susedi većinski pripadaju drugoj klasi, outliere koji se nalaze izolovano daleko od svoje klase i ne prate distribuciju drugih instanci iste klase, i ambiguoze primere koji se nalaze u preklapajućim oblastima različitih klasa gde je granica odlučivanja nejasna ili gde različite klase imaju sličnu distribuciju.

Za razliku od kondenzacije koja agresivno uklanja internal instances koje čine većinu skupa, edition metode su konzervativnije i fokusiraju se na čišćenje graničnih oblasti između klasa, uklanjajući samo instance za koje postoji jaka indicija da predstavljaju šum. Tipična redukcija kod edition pristupa je umerena, obično između 10-30% zavisno od kvaliteta originalnih podataka, ali sa potencijalnim poboljšanjem tačnosti za 2-5 procentnih poena u odnosu na originalni skup, što je značajno jer se istovremeno i smanjuje veličina i povećava kvalitet.

Edited Nearest Neighbor (ENN) algoritam razvijen od strane Dennis L. Wilson-a 1972. godine i njegove varijante (Repeated ENN koji iterativno primenjuje ENN dok god dođe do promena, i All k-NN koji koristi različite vrednosti parametra k) su reprezentativni primeri ove kategorije.

Edition je naročito važan u scenarijima gde podaci potiču iz nesigurnih izvora kao što su crowdsourcing platforme gde anotatori mogu praviti greške, automatski sistemi za etiketiranje koji nisu savršeno tačni, ili senzori koji povremeno daju pogrešna očitavanja, kada postoji sumnja na kvalitet etiketiranja zbog nedovoljno obučених anotatora ili nejasnih definicija klasa, ili kada je model u produkciji i potrebno je održavanje kvaliteta podataka tokom vremena kako se akumuliraju novi primeri koji mogu uključivati šum.

Hibridni pristupi

Hibridni pristupi kombinuju više ciljeva istovremeno, tipično kombinujući edition za inicijalno uklanjanje šuma sa condensation za naknadnu redukciju veličine, pokušavajući da iskoriste prednosti obe strategije dok minimizuju njihove nedostatke. Najpopularniji hibridni algoritam je DROP3 koji prvo primenjuje ENN za čišćenje šumnih instanci što kreira čistiji skup podataka kao polaznu tačku, a zatim DROP2 algoritam za kondenzaciju koji dekrimentalno uklanja instance počevši od onih koje su najmanje kritične za klasifikaciju svojih suseda, postižući balans između kvaliteta podataka i efikasnosti reprezentacije.

Eksperimentalni rezultati na benchmark skupovima pokazuju da DROP3 često postiže najbolje rezultate u praksi, sa redukcijom od 80-90% uz zadržavanje ili čak blagim poboljšanjem tačnosti u odnosu na originalni skup, što je superiornije u odnosu na primenu samo kondenzacije ili samo edition metoda. Ovi pristupi su računski najzahtevniji jer kombinuju dve faze obrade, svaka sa svojom složenošću, što rezultuje u ukupnoj složenosti $O(n^3)$ ili $O(n^4)$, ali taj trošak se isplati kroz superiornije konačne rezultate.

Dodatno, specifični ciljevi mogu uključivati balansiranje nebalansiranih klasa kroz targetirano selektivno uklanjanje instanci iz većinske klase pri čemu se zadržavaju sve kritične instance iz manjinske klase, optimizaciju za specifične tipove klasifikatora gde na primer SVM može imati drugačije zahteve fokusirajući se na support vectors dok k-NN fokusira na lokalne komšiluke, ili prilagođavanje distribucije podataka za bolje pokrivanje feature space-a identifikovanjem i popunjavanjem regiona sa malom gustinom podataka.

Izbor odgovarajućeg cilja i metode zavisi od pažljive analize trade-off-a između konkurentnih faktora, karakteristika konkretnog skupa podataka uključujući veličinu, dimenzionalnost, stepen šuma i balansiranost klasa, i prioriteta konkretne aplikacije u pogledu tačnosti, brzine, memorije i interpretabilnosti.

3. ALGORITMI ZA SELEKCIJU INSTANCI

Metode selekcije instanci mogu se klasifikovati prema različitim kriterijumima, pri čemu najčešća podela obuhvata četiri glavne kategorije algoritama: metode kondenzacije koje agresivno smanjuju veličinu trening skupa fokusirajući se na granice između klasa, metode uređivanja koje prioritizuju kvalitet podataka kroz uklanjanje šuma i outliera, hibridne metode koje kombinuju oba pristupa kako bi postigle balans između redukcije i kvaliteta, i evolucione metode koje koriste meta-heuristike za optimizaciju selekcije.

Svaka od ovih kategorija razvijala se tokom decenija istraživanja, počevši od pionirskog rada Hart-a 1968. godine na Condensed Nearest Neighbor algoritmu, preko Wilson-ovog Edited Nearest Neighbor algoritma iz 1972. godine, do savremenih sofisticiranih pristupa kao što su DROP familija algoritama i genetski algoritmi. Razumevanje ovih metoda, njihovih prednosti i ograničenja, kao i konteksta u kome su najefikasnije, ključno je za pravilnu primenu selekcije instanci u praktičnim aplikacijama.

3.1 Metode kondenzacije

Metode kondenzacije predstavljaju najstariju i najproučavaniju kategoriju algoritama za selekciju instanci, sa primarnim ciljem da drastično smanje veličinu trening skupa uz minimalan uticaj na tačnost klasifikacije. Osnovna ideja ovih metoda zasniva se na zapažanju da većina instanci u tipičnom skupu podataka ne doprinosi definisanju granica odlučivanja između klasa, već se nalazi duboko unutar regiona svoje klase gde njihovo prisustvo ili odsustvo ne utiče na klasifikaciju novih primera.

Ove internal instances čine često 80-95% trening skupa, što znači da njihovo uklanjanje može dovesti do dramatične redukcije veličine bez gubitka informacija relevantnih za klasifikaciju. Nasuprot tome, boundary instances koje se nalaze blizu granica između klasa su kritične jer direktno definišu gde prolazi granica odlučivanja, i njihovo uklanjanje bi moglo značajno degradirati performanse. Metode kondenzacije pokušavaju da identifikuju i zadrže samo boundary instances, kreirajući minimalni ili skoro-minimalni podskup koji održava konzistentnost klasifikacije sa originalnim skupom.

Condensed Nearest Neighbor (CNN)

Condensed Nearest Neighbor algoritam, koji je razvio Hart 1968. godine, predstavlja prvi formalni pristup kondenzaciji skupa podataka i postavlja osnovu za mnoge kasnije razvijene metode. Algoritam funkcioniše kroz iterativni rastući proces gde se skup S kondenzovanih instanci gradi postepeno dodavanjem kritičnih primera.

Na početku, S se inicijalizuje sa po jednom nasumično izabranom instancom iz svake klase, što garantuje da svaka klasa ima barem jednog predstavnika u kondenzovanom skupu. Zatim algoritam iterativno prolazi kroz sve instance iz originalnog skupa T , klasifikujući svaku pomoću trenutnog skupa S sa 1-NN pravilom. Kada se nađe na instancu koja je pogrešno klasifikovana od strane trenutnog S , ta instanca se dodaje u S

jer njena pogrešna klasifikacija ukazuje da se nalazi na granici između klasa ili blizu nje i da je potrebna za pravilno definisanje te granice.

Proces se nastavlja dok god postoje instance iz T koje su pogrešno klasifikovane, što garantuje da finalni skup S pravilno klasifikuje sve instance iz T , čime se održava konzistentnost sa originalnim skupom. Tipična redukcija koju CNN postiže kreće se od 70% do 95% zavisno od prirode podataka, pri čemu kompaktniji klasteri sa jasno definisanim granicama dozvoljavaju veću redukciju, dok složeniji skupovi sa isprepletanim klasama zahtevaju više boundary instanci.

Karakteristike CNN algoritma

Ključna karakteristika CNN algoritma je njegova osetljivost na redosled procesiranja instanci, što znači da različiti redosledi mogu dovesti do različitih finalnih skupova S čak i na istim podacima. Ovo proizilazi iz činjenice da koja instanca će biti dodata u S zavisi od toga koje instance su već u S u trenutku kada se ta instanca obrađuje. Na primer, ako se instance obrade u jednom redosledu, određena instanca može biti pogrešno klasifikovana i dodata u S , dok u drugom redosledu, druge instance koje su već u S mogu pravilno klasifikovati tu istu instancu što znači da neće biti dodata.

Ova nestabilnost može biti problematična za reproducibilnost rezultata, ali se može ublažiti korišćenjem fiksnog semena za randomizaciju ili deterministički definisanim redosledom. Vremenska složenost CNN algoritma je $O(n^3)$ u najgorem slučaju, što proizilazi iz činjenice da u svakoj iteraciji (kojih može biti $O(n)$) potrebno je klasifikovati sve instance ($O(n)$) koristeći trenutni skup S (što zahteva $O(n)$ operacija u najgorem slučaju).

Ova kubna složenost može biti prohibitivna za velike skupove podataka sa milionima instanci, što je motivisalo razvoj efikasnijih varijanti. Prednosti CNN algoritma uključuju jednostavnost implementacije koja zahteva samo osnovnu 1-NN klasifikaciju bez složenih parametara ili optimizacija, garantovanu konzistentnost gde finalni skup S pravilno klasifikuje sve instance iz T , i dramatičnu redukciju veličine koja može značajno ubrzati klasifikaciju novih instanci.

Glavne mane su već pomenuta osetljivost na redosled, visoka vremenska složenost, i činjenica da CNN ne uklanja šum već može čak i zadržati šumne instance ako one padnu na pogrešnu stranu granice i budu pogrešno klasifikovane.

Reduced Nearest Neighbor (RNN)

Reduced Nearest Neighbor algoritam, koji je predložio Gates 1972. godine, predstavlja unapređenje CNN algoritma sa ciljem da dodatno redukuje broj instanci uklanjanjem redundantnih boundary instances koje CNN može zadržati. Osnovna motivacija za RNN proizilazi iz zapažanja da CNN često zadržava više boundary instanci nego što je striktno neophodno za održavanje konzistentnosti, jer inkrementalni proces dodavanja može rezultovati u situaciji gde više instanci pokrivaju istu oblast granice odlučivanja.

RNN započinje sa izlazom CNN algoritma kao polaznom tačkom, što znači da već ima relativno mali skup S sa visokom redukcijom, a zatim pokušava da dalje smanji taj skup uklanjanjem instanci koje se pokazuju kao nepotrebne. Algoritam funkcioniše tako što iterativno prolazi kroz svaku instancu s u trenutnom skupu S , privremeno je uklanja kreirajući skup $S' = S \setminus \{s\}$, i zatim proverava da li skup S' još uvek pravilno klasifikuje sve instance iz originalnog skupa T korišćenjem 1-NN pravila.

Ako je S' još uvek konzistentan sa T , to znači da instanca s nije bila kritična i može biti trajno uklonjena iz S . Ako uklanjanje s dovodi do pogrešne klasifikacije bar jedne instance iz T , tada se s zadržava u S jer je njena prisutnost neophodna za održavanje konzistentnosti. Proces se nastavlja kroz sve instance u S , pri čemu se u svakoj iteraciji skup potencijalno smanjuje, sve dok se ne pronađe minimalni podskup koji održava konzistentnost ili dok više nijedna instanca ne može biti uklonjena bez narušavanja konzistentnosti.

RNN tipično postiže dodatnih 10-30% redukcije u odnosu na CNN, što znači da ako CNN redukuje skup na 20% originalne veličine, RNN može dalje redukovati na 14-18%. Ova dodatna redukcija nije dramatična kao kod CNN, ali je značajna jer se postiže bez gubitka tačnosti, a svaka preostala instanca u skupu doprinosi pravilnoj klasifikaciji bar jednog primera, pod uslovom da algoritam konvergira ka globalnom optimumu.

Vremenska složenost RNN algoritma je $O(n^4)$ jer za svaku od $O(n)$ instanci u trenutnom S potrebno je proveriti konzistentnost što zahteva klasifikaciju svih $O(n)$ instanci iz T korišćenjem $O(n)$ preostalih instanci u S' , što daje $O(n) \cdot O(n) \cdot O(n) = O(n^3)$ za jednu iteraciju, a može biti potrebno $O(n)$ iteracija pre konvergencije. Ova kvartna složenost čini RNN praktično neupotrebljivim za skupove sa više od nekoliko hiljada instanci, što je ozbiljno ograničenje u eri velikih skupova podataka.

Prednosti RNN uključuju manju veličinu finalnog skupa u odnosu na CNN što dodatno ubrzava klasifikaciju, teorijski minimalni ili skoro-minimalni konzistentni skup što je poželjno sa stanovišta kompresije podataka, i garanciju da nijedna instanca u finalnom skupu nije redundantna jer svaka doprinosi pravilnoj klasifikaciji bar jedne instance iz T . Glavni nedostaci su veoma visoka vremenska složenost koja je praktično prohibitivna za velike skupove, i dalje prisutna osetljivost na redosled iako u manjoj meri nego kod CNN.

Minimal Subset Selection (MSS)

Minimal Subset Selection predstavlja teorijski optimalan pristup kondenzaciji gde je cilj naći apsolutno najmanji podskup $S \subseteq T$ koji pravilno klasifikuje sve instance iz T . Ovaj problem je fundamentalno različit od pristupa CNN i RNN jer eksplicitno formuliše optimizacioni cilj kao minimizaciju kardinaliteta, dok CNN i RNN koriste greedy heuristike koje ne garantuju optimalnost.

MSS problem je dokazano NP-hard, što znači da ne postoji algoritam koji može efikasno naći optimalno rešenje za sve moguće skupove podataka u polinomijalnom vremenu osim ako nije $P=NP$, što je jedan od najvećih nerešenih problema u računarstvu. Ova

NP-hardness proizilazi iz toga što je potrebno razmotriti eksponencijalno mnogo mogućih podskupova (2^n) i za svaki proveriti konzistentnost, što čak i za umereno velike n (npr. $n=100$) postaje praktično nemoguće.

Zbog toga se u praksi koriste aproksimativni i heuristički algoritmi koji pokušavaju da nađu dovoljno dobra rešenja bez garancije optimalnosti. Jedan pristup su genetski algoritmi koji kodiraju svaki kandidatni podskup kao binarni string dužine n gde 1 označava da instanca pripada podskupu a 0 da ne pripada, i zatim koriste evolucione operatore (selekcija, crossover, mutacija) da pretražuju prostor mogućih rešenja vođeni fitness funkcijom koja penalizuje veće podskupove i nekonzistentnost.

Drugi pristup je lokalna pretraga koja počinje sa nekim početnim rešenjem (npr. izlaz CNN-a) i iterativno pokušava da poboljša rešenje uklanjanjem instanci ili zamenom instanci sa ciljem smanjenja veličine uz očuvanje konzistentnosti. Treći pristup su branch-and-bound metode koje sistematski pretražuju prostor podskupova ali koriste bounds da obrežu grane pretrage koje ne mogu dovesti do boljeg rešenja od trenutno najboljeg.

Iako ovi pristupi ne garantuju nalaženje globalnog optimuma, u praksi često nalaze rešenja koja su blizu optimuma ili čak optimalna za mnoge skupove podataka. Značaj MSS problema je više teorijski nego praktičan jer postavlja donju granicu na to koliko mala kondenzacija može biti, i pruža benchmark za evaluaciju heurističkih algoritama kao što su CNN i RNN. U praksi, razlika između veličine koju CNN postiže i teorijski minimalnog skupa često nije velika (obično 10-30%), što znači da jednostavniji i brži CNN često daje prihvatljivo dobra rešenja bez potrebe za skupom optimizacijom.

3.2 Metode uređivanja

Metode uređivanja zauzimaju fundamentalno drugačiji pristup problemu selekcije instanci u odnosu na metode kondenzacije, fokusirajući se primarno na poboljšanje kvaliteta podataka kroz identifikaciju i uklanjanje šumnih, pogrešno označenih i atipičnih instanci, dok redukcija veličine skupa predstavlja sekundarni cilj koji nastaje kao posledica čišćenja.

Osnovna motivacija za edition metode proizilazi iz zapažanja da realni skupovi podataka retko sadrže savršeno čiste podatke, već skoro uvek uključuju izvestan procenat šuma koji može nastati iz brojnih izvora: ljudske greške tokom ručnog etiketiranja gde označivači pogrešno klasifikuju nejasne ili granične slučajeve, automatski sistemi za prikupljanje koji nisu savršeno tačni kao što su senzori sa povremenim kvarovima ili algoritmi za automatsko etiketiranje sa limitiranom tačnošću, promene u definiciji klasa tokom vremena što dovodi do toga da stari primeri postaju nekonzistentni sa novim definicijama, ili outlieri koji predstavljaju retke ekstremne slučajeve koji ne prate distribuciju svoje klase.

Prisustvo čak i malog procenta šuma (5-10%) može značajno degradirati performanse k-NN klasifikatora jer takvi algoritmi ne prave eksplicitnu generalizaciju već se direktno

oslanjaju na trening instance, što znači da svaka šumna instanca može dovesti do pogrešne klasifikacije novih primera koji padnu u njenu blizinu.

Edited Nearest Neighbor (ENN)

Edited Nearest Neighbor algoritam, koji je predložio Wilson 1972. godine, predstavlja najpoznatiju i najčešće korišćenu edition metodu. Algoritam se zasniva na jednostavnoj ali moćnoj ideji: instanca koja je pogrešno klasifikovana od strane svojih najbližih suseda verovatno predstavlja šum ili je pogrešno označena, i stoga treba da bude uklonjena iz trening skupa.

ENN funkcioniše kroz jedan prolaz kroz sve instance u skupu, pri čemu se za svaku instancu (x, y) pronalazi njenih k najbližih suseda u preostalom delu skupa (isključujući samu x), klasifikuje se x na osnovu većinskog glasanja tih k suseda, i ako je predviđena klasa različita od stvarne oznake y , instanca se označava za uklanjanje. Nakon što se sve instance obrade, one označene za uklanjanje se trajno eliminišu iz skupa.

Uticaj parametra k

Parametar k , koji određuje koliko suseda se razmatra, ima kritičan uticaj na ponašanje algoritma i predstavlja trade-off između agresivnosti čišćenja i konzervativnosti. Za $k=1$ algoritam je najagresivniji i uklanja svaku instancu čiji najbliži sused pripada drugoj klasi, što može ukloniti mnogo instanci uključujući validne boundary instances koje leže blizu granice između klasa ali su ispravno označene.

Za $k=3$, što je najčešće korišćena vrednost u praksi, algoritam je balansirani i zahteva da bar 2 od 3 najbliža suseda pripadaju istoj klasi, što pruža određenu toleranciju na lokalne nehomogenosti u distribuciji ali i dalje uklanja očigledne outliere i pogrešne oznake. Za $k \geq 5$ algoritam postaje veoma konzervativan i uklanja samo instance koje su jasno izolovane ili duboko unutar teritorije druge klase, što može biti premalo agresivno za efikasno čišćenje šuma.

Tipična redukcija koju ENN postiže je umerena, obično između 10-30% zavisno od kvaliteta originalnih podataka i izbora parametra k , što je značajno manje nego kod CNN koji postiže 70-95% redukcije. Međutim, ključna razlika je da ENN često dovodi do poboljšanja tačnosti klasifikacije na novim podacima, sa tipičnim povećanjem od 2-5 procentnih poena, jer uklanjanje šuma omogućava klasifikatoru da preciznije nauči prave granice između klasa bez distraktora.

Vremenska složenost ENN algoritma je $O(n^2 \cdot d)$ gde je n broj instanci a d broj dimenzija, što proizilazi iz činjenice da za svaku od n instanci potrebno je naći k najbližih suseda što zahteva računanje udaljenosti do svih ostalih $n-1$ instanci ($O(n)$), pri čemu svako računanje udaljenosti u d -dimenzionalnom prostoru zahteva $O(d)$ operacija. Ova kvadratna složenost po n čini ENN značajno bržim od CNN ($O(n^3)$) i RNN ($O(n^4)$), što ga čini praktično upotrebljivim čak i za relativno velike skupove podataka.

Prednosti i ograničenja ENN-a

Prednosti ENN uključuju poboljšanje generalizacione sposobnosti modela kroz uklanjanje šuma što često rezultuje u boljim performansama na test podacima, jednostavnost algoritma sa samo jednim kritičnim parametrom k koji se lako podešava, stabilnost rezultata jer algoritam nije osetljiv na redosled instanci, i relativno niska vremenska složenost u poređenju sa metodama kondenzacije.

Glavni nedostaci su potencijalno uklanjanje validnih boundary instances koje leže blizu granice i mogu biti pogrešno klasifikovane od svojih suseda iako su ispravno označene, umerena redukcija koja ne rešava problem velikih skupova podataka gde je cilj drastično smanjenje veličine, i osetljivost na izbor parametra k gde loš izbor može biti ili preterano agresivan ili preterano konzervativan.

Repeated ENN (RENN)

Repeated ENN predstavlja iterativnu varijantu originalnog ENN algoritma koja rešava ograničenje da ENN vrši samo jedan prolaz kroz podatke, što može biti nedovoljno jer uklanjanje nekih instanci u prvom prolazu može otkriti dodatne šumne instance koje nisu bile detektovane dok su te instance bile prisutne. Na primer, ako postoji klaster od nekoliko pogrešno označenih instanci, one mogu međusobno biti najbliži susedi jedna drugoj i stoga neće biti uklonjene u prvom prolazu jer ih njihovi susedi (koji su isto pogrešno označeni) podržavaju.

Međutim, nakon što se uklone instance van tog klastera koje su detektovane kao šum, klaster pogrešnih instanci postaje izolovan i u sledećem prolazu može biti identifikovan i uklonjen. RENN funkcioniše tako što iterativno primenjuje ENN algoritam, čuvajući skup S iz prethodne iteracije kao S_{prev} , primenjujući ENN na trenutni S da dobije novi S , i poređujući da li je $S = S_{prev}$ što bi značilo da više nijedna instanca nije uklonjena. Proces se nastavlja dok god se S menja, odnosno dok god se u svakoj iteraciji ukloni bar jedna instanca, i terminira kada S stabilizuje što garantuje da su sve šumne instance koje mogu biti detektovane uz dati k uklonjene.

Broj iteracija koje RENN zahteva zavisi od strukture šuma u podacima, pri čemu izolovani outlieri se uklanjaju u prvoj iteraciji, klasteri šuma zahtevaju više iteracija (obično 2-4) gde se postepeno "ljušte" spoljašnje instance klastera dok se ne uklone sve, a složene strukture sa isprepletanom šumom mogu zahtevati više iteracija (5-10). U praksi, većina skupova podataka konvergira za 2-5 iteracija.

RENN tipično postiže dodatnih 5-15% redukcije u odnosu na jedan prolaz ENN-a, što znači da ako ENN ukloni 20% instanci, RENN može ukloniti ukupno 25-35%.

Vremenska složenost RENN algoritma je $O(k \cdot n^2 \cdot d)$ gde je k broj iteracija, što znači da je linearno spora po broju iteracija ali kvadratna po broju instanci kao i običan ENN.

Na skupovima gde ENN već u prvom prolazu ukloni sve detektabilne šumne instance, RENN konvergira u jednoj iteraciji i daje identičan rezultat. Ovo se može desiti kada su šumne instance izolovane, a ne grupisane u klastere koji bi zahtevali višestruke prolaze.

All K-NN

All k-NN predstavlja konzervativniju varijantu ENN algoritma koja kombinuje multiple vrednosti parametra k kako bi identifikovala samo one instance za koje postoji jak konsenzus da predstavljaju šum. Umesto korišćenja jedne fiksne vrednosti k , All k-NN primenjuje ENN sa više različitih vrednosti k (tipično $k = 1, 2, 3, \dots, k_{\text{max}}$) i zadržava samo one instance koje prežive sve iteracije.

Formalno, instanca se uklanja samo ako je pogrešno klasifikovana za sve testirane vrednosti k , što znači da mora postojati jak signal da je ta instanca problematična bez obzira na lokalnu okruženje. Algoritam tipično koristi k_{max} između 5 i 10 zavisno od veličine skupa podataka, pri čemu se za svaku testiranu vrednost k_i vrši nezavisna evaluacija da li je instanca pravilno klasifikovana od strane svojih k_i najbližih suseda, i instanca se označava za uklanjanje samo ako padne na testu za sve vrednosti.

Ovaj pristup je motivisan zapažanjem da pogrešno označene instance i pravi outlieri će biti pogrešno klasifikovani nezavisno od toga da li posmatramo 1, 3, 5 ili 10 najbližih suseda jer ne postoji lokalna podrška njihovoj oznaci, dok validne boundary instances mogu biti pogrešno klasifikovane sa $k=1$ ili $k=3$ jer leže blizu granice, ali pravilno klasifikovane sa većim k jer postoji dovoljna podrška u široj okolini.

All k-NN postiže najmanju redukciju među edition metodama, tipično samo 5-15%, jer zadržava sve instance osim onih za koje postoji veoma jak signal da su problematične. Ova konzervativnost čini metodu bezbednom u smislu da minimizuje rizik od uklanjanja validnih instanci, ali možda ne uklanja sve šumne instance posebno one koje imaju lokalnu podršku u vidu nekoliko drugih šumnih instanci u blizini.

Tomek Links

Tomek Links predstavlja specifičan pristup uređivanju koji je predložio Ivan Tomek 1976. godine, zasnovan na identifikaciji parova instanci različitih klasa koje su međusobno najbliži susedi. Za razliku od ENN-a koji razmatra k suseda svake instance i donosi odluku na osnovu većinskog glasanja, Tomek Links se fokusira isključivo na parove instanci koje se nalaze na samoj granici između klasa.

Formalno, par instanci (a, b) čini Tomek link ako instanca a pripada jednoj klasi a instanca b drugoj, i ne postoji nijedna treća instanca c koja je bliža bilo kojoj od njih od njihovog međusobnog rastojanja. Ovakav par ukazuje da se te dve instance nalaze na granici odlučivanja ili da je jedna od njih šumna instanca koja je zalutala u teritoriju druge klase. Nakon identifikacije svih Tomek linkova, algoritam uklanja instancu iz para koja pripada većinskoj klasi, čime se istovremeno čisti granica između klasa i blago ublažava nebalansiranost.

Ključna razlika u odnosu na ostale edition metode je u tome što Tomek Links ne razmatra širu okolinu instance već isključivo njen odnos sa najbližim susedom druge klase. Ovo čini metodu izuzetno konzervativnom jer se uklanjaju samo instance koje su u direktnom konfliktu sa instancom suprotne klase, dok se sve ostale instance zadržavaju bez obzira na to koliko su blizu granici. Tipična redukcija koju Tomek Links postiže je 5-15%, što je

najmanje među svim metodama uređivanja, ali sa minimalnim rizikom od gubitka korisnih instanci.

Vremenska složenost je $O(n^2 \cdot d)$ za pronalaženje svih parova najbližih suseda, što je uporedivo sa ENN-om. U praksi se Tomek Links često koristi kao korak predobrade pre primene drugih metoda, ili u kombinaciji sa oversampling tehnikama poput SMOTE gde SMOTE generiše nove instance manjinske klase a Tomek Links naknadno čisti granice između klasa kako bi se smanjilo preklapanje.

3.3 Hibridne metode

Hibridne metode predstavljaju evoluciju u razvoju algoritama za selekciju instanci, nastale iz zapažanja da čiste metode kondenzacije i čiste metode uređivanja imaju komplementarne prednosti i mane koje se mogu efikasno kombinovati. Metode kondenzacije postižu dramatičnu redukciju ali ne obraćaju posebnu pažnju na kvalitet podataka i mogu čak zadržati šumne instance ako one padnu na granicu između klasa, dok metode uređivanja efikasno čiste šum ali ne postižu značajnu redukciju veličine.

Hibridni pristupi pokušavaju da iskoriste najbolje od oba sveta kombinujući edition i condensation u multi-faze algoritme gde se najpre primenjuje edition za kreiranje čistog polaznog skupa, a zatim condensation za agresivnu redukciju veličine na tom čistom skupu. Ovaj pristup ne samo da daje bolje konačne rezultate od primene bilo koje pojedinačne metode, već i omogućava da condensation algoritmi rade na kvalitetnijim podacima što povećava verovatnoću da će identifikovati prave boundary instances umesto šumnih instanci koje se slučajno nalaze na granici.

DROP algoritmi

DROP (Decremental Reduction Optimization Procedure) familija algoritama predstavlja sofisticiran pristup selekciji instanci koji kombinuje edition i condensation kroz decremental proces uklanjanja instanci uz pažljivo razmatranje uticaja svakog uklanjanja na klasifikaciju ostalih instanci. Ključni koncept koji DROP uvodi je ideja associates (pridružene instance), gde se za svaku instancu definiše skup njenih associates kao skup svih instanci za koje je ona jedan od k najbližih suseda.

Formalno, instanca y je pridružena instanca x ako je $x \in NN_k(y)$ gde je $NN_k(y)$ skup k najbližih suseda od y . Ovaj koncept omogućava razumevanje koliko je neka instanca važna za pravilnu klasifikaciju drugih instanci: ako instanca x ima mnogo pridruženih instanci, to znači da je ona blizu mnogim drugim instancama i njena prisutnost ili odsustvo može uticati na njihovu klasifikaciju, dok instanca sa malo pridruženih instanci je relativno izolovana i njena uklanjanje neće uticati na mnoge druge instance.

DROP3 algoritam

DROP3 kombinuje edition i condensation u dvo-fazni pristup koji je postao najuspešnija i najčešće korišćena varijanta DROP familije. U prvoj fazi, DROP3 primenjuje ENN (Edited Nearest Neighbor) na originalni skup podataka kako bi uklonio očigledan šum, pogrešno označene instance i outliere, kreirajući čišći skup S_{clean} koji

služi kao polazna tačka za drugu fazu. Ovo inicijalno čišćenje je kritično jer osigurava da condensation faza radi na kvalitetnim podacima i neće greškom identifikovati šumne instance kao boundary instances koje treba zadržati.

U drugoj fazi, DROP3 primenjuje DROP2 algoritam na S_{clean} , postižući agresivnu redukciju veličine dok održava kvalitet klasifikacije. DROP2 ne sortira instance već ih razmatra u nekom arbitrarnom redosledu, i koristi koncept pokrivenosti (coverage) gde se evaluira koliko je instanca korisna za klasifikaciju svojih pridruženih instanci. Za svaku instancu p , algoritam računa koliko njenih instanci bi bilo pogrešno klasifikovano ako se p ukloni.

Ako nijedna pridružena instanca ne bi bila pogrešno klasifikovana, ili ako se broj pogrešno klasifikovanih pridruženih instanci ne poveća, p se može bezbedno ukloniti. Dodatno, DROP2 koristi dinamičko ažuriranje gde nakon svakog uklanjanja instance p , lista najbližih suseda i pridruženih instanci za sve ostale instance se ažurira da reflektuje novo stanje, što omogućava da kasnije odluke uzmu u obzir već izvršena uklanjanja.

Performanse DROP3

Empirijski rezultati na velikom broju benchmark skupova podataka pokazuju da DROP3 konstantno nadmašuje i DROP1 i DROP2 u pogledu balansa između redukcije i tačnosti. Tipična redukcija koju DROP3 postiže kreće se od 80% do 90%, što je uporedivo ili bolje od CNN-a, ali sa značajnom prednošću što DROP3 često postiže istu ili bolju tačnost od originalnog skupa zahvaljujući inicijalnom čišćenju šuma.

Na primer, na benchmark skupovima kao što su Iris, Wine, ili Ionosphere, DROP3 može redukovati skup na 10-15% originalne veličine uz poboljšanje tačnosti za 1-3 procentna poena u odnosu na korišćenje celokupnog skupa. Vremenska složenost DROP algoritama je $O(n^3)$ što proizilazi iz činjenice da za svaku od n instanci potrebno je evaluirati uticaj na njene associates, pri čemu svaka evaluacija zahteva k -NN pretragu koja u naivnoj implementaciji zahteva $O(n)$ vremena, i ovo se ponavlja za $O(n)$ instanci.

Ova kubna složenost čini DROP algoritme računski zahtevnijima od ENN ($O(n^2)$) ali uporedljivima sa CNN-om. Međutim, u praksi DROP algoritmi mogu biti optimizovani korišćenjem efikasnih struktura podataka kao što su KD-trees ili Ball trees za k -NN pretragu, što smanjuje prosečnu složenost k -NN sa $O(n)$ na $O(\log n)$, rezultujući u ukupnoj $O(n^2 \log n)$ složenosti što je značajno brže za velike skupove.

Važno je napomenuti da ovakvi rezultati pretpostavljaju skupove podataka sa relativno jasno definisanim granicama između klasa. Na skupovima sa izraženim preklapanjem, DROP3 može postići veću stopu redukcije od tipičnih 80-90%, ali uz značajniji pad tačnosti, jer se u procesu kondenzacije gube instance koje nose informaciju o nedovoljno jasnim granicama.

Iterative Case Filtering (ICF)

Iterative Case Filtering predstavlja alternativni hibridni pristup koji su razvili Henry Brighton i Chris Mellish 1999. godine, a koji koristi koncepte dostižnosti i pokrivenosti za identifikaciju instanci koje mogu biti uklonjene. ICF se zasniva na zapažanju da instance u skupu podataka mogu se klasifikovati u nekoliko kategorija prema njihovoj poziciji i ulozi: unutrašnje instance koje su duboko unutar svoje klase i nisu blizu granica, granične instance koje se nalaze na ili blizu granica između klasa, i šumne instance koje su izolovane ili okružene instancama druge klase.

Cilj ICF-a je da identifikuje i ukloni unutrašnje instance koje su redundantne, dok zadržava granične instance koje su kritične za definisanje granica, i potencijalno ukloni šumne instance. Algoritam uvodi dva ključna koncepta: skup dostižnosti $R(x)$ instance x je skup svih instanci iste klase koje mogu biti dostignute od x kroz lanac k najbližih suseda koji svi pripadaju istoj klasi, što meri koliko je instanca povezana sa svojom klasom, a skup pokrivenosti $C(x)$ je skup instanci suprotne klase za koje je x jedan od k najbližih suseda, što meri koliko je instanca važna za klasifikaciju instanci drugih klasa.

Algoritam funkcioniše kroz dva glavna koraka. U prvom koraku, ICF izračunava skupove dostižnosti i pokrivenosti za sve instance u skupu, što zahteva konstrukciju povezanog grafa gde su čvorovi instance a ivice postoje između k najbližih suseda iste klase. U drugom koraku, algoritam identifikuje instance za uklanjanje korišćenjem pravila: instanca x je kandidatna za uklanjanje ako ima veliki skup dostižnosti ($|R(x)| > \text{threshold_R}$), što znači da je dobro povezana sa svojom klasom i verovatno unutrašnja instanca, i ima mali skup pokrivenosti ($|C(x)| < \text{threshold_C}$), što znači da nije kritična za razlikovanje od drugih klasa.

ICF tipično postiže redukciju od 60-80% sa dobrim očuvanjem ili blagim poboljšanjem tačnosti, što je uporedivo sa DROP3 algoritmom. Vremenska složenost ICF-a je $O(n^2 \cdot k)$ za konstrukciju grafova i računanje skupova dostižnosti i pokrivenosti, plus $O(n)$ za identifikaciju instanci za uklanjanje, što daje ukupnu $O(n^2 \cdot k)$ složenost. Ovo je uporedivo sa ENN ali potencijalno brže od DROP algoritama zavisno od implementacije.

3.4 Evolucionarne metode

Evolucionarne metode predstavljaju fundamentalno drugačiji pristup selekciji instanci u odnosu na determinističke algoritme kao što su CNN, ENN ili DROP, tretirajući problem kao globalni optimizacioni zadatak koji se rešava meta-heurističkim algoritmima inspirisanim prirodnim procesima. Osnovna motivacija za primenu evolucionih metoda proizilazi iz činjenice da je problem nalaženja optimalnog podskupa instanci NP-hard, što znači da deterministički algoritmi mogu zapeti u lokalnim optimumima ili zahtevati neprihvatljivo dugo vreme za nalaženje globalnog optimuma.

Evolucionarni algoritmi ne garantuju nalaženje globalnog optimuma, ali mogu efikasno istražiti veliki prostor mogućih rešenja kroz stohastičku pretragu vođenu principima

selekcije, reprodukcije i mutacije, često pronalazeći vrlo kvalitetna rešenja koja nadmašuju pohlepne heuristike. Dodatna prednost evolucionih pristupa je njihova fleksibilnost u definisanju funkcije cilja: umesto da se oslanjaju na unapred definisane heuristike kao što je "zadrži granične instance", evolucioni algoritmi mogu direktno optimizovati bilo koju kombinaciju ciljeva kao što su tačnost klasifikacije, stopa redukcije, balans između klasa, ili čak funkcije sa više ciljeva koje istovremeno razmatraju više kriterijuma.

Genetski algoritmi (GA)

Genetski algoritmi predstavljaju najpoznatiju i najčešće primenjivanu evolucionu tehniku za selekciju instanci, inspirirani Darwinovom teorijom evolucije kroz prirodnu selekciju. U kontekstu selekcije instanci, genetski algoritam kodira svaki kandidat podskup kao hromozom, predstavljen kao binarni string dužine n gde je n broj instanci u originalnom skupu T . Svaka pozicija i u hromozomu odgovara i -toj instanci u T , pri čemu vrednost 1 označava da instanca pripada podskupu S , a vrednost 0 da ne pripada.

Algoritam započinje sa inicijalizacijom populacije od P hromozoma (tipično $P = 50-200$), gde se svaki hromozom kreira nasumičnim postavljanjem svakog bita na 1 sa nekom verovatnoćom p_{init} (obično 0.3-0.5). Nakon inicijalizacije, algoritam ulazi u iterativni proces evolucije kroz G generacija (tipično 50-500). U svakoj generaciji, svaki hromozom se evaluira fitness funkcijom koja meri kvalitet reprezentovanog podskupa.

Najčešće korišćena fitness funkcija je kombinacija tačnosti klasifikacije i stope redukcije: $fitness(S) = \alpha \cdot Accuracy(S) + (1-\alpha) \cdot (1-|S|/|T|)$, gde $\alpha \in [0,1]$ balansira važnost tačnosti protiv redukcije. Evaluacija tačnosti tipično se vrši kroz cross-validation na trening skupu, gde se reprezentovani podskup S koristi za treniranje k -NN klasifikatora koji se zatim testira na validacionom delu podataka.

Nakon evaluacije svih hromozoma, pristupa se selekciji roditelja za reprodukciju korišćenjem tournament selection ili roulette wheel selection. Zatim se primenjuje crossover operator koji kombinuje dva roditeljska hromozoma da kreira potomke, i mutacija sa malom verovatnoćom p_{mut} (tipično 0.01-0.05) gde se svaki bit u hromozomu nasumično invertuje. Algoritam se nastavlja kroz G generacija ili dok se ne postigne konvergencija.

Genetski algoritmi tipično postižu redukciju od 70-85% uz tačnost uporedivu ili bolju od CNN ili DROP3, zavisno od podešavanja parametara i broja generacija. Vremenska složenost GA je $O(G \cdot P \cdot n \cdot T_{eval})$ gde je G broj generacija, P veličina populacije, n broj instanci, i T_{eval} vreme potrebno za evaluaciju jednog hromozoma kroz cross-validation, što može biti prohibitivno za velike skupove.

Particle Swarm Optimization (PSO)

Particle Swarm Optimization predstavlja alternativnu evolucionu tehniku inspirisanu socijalnim ponašanjem jata ptica ili riba, gde individue (čestice) teže ka svojim ličnim najboljim pozicijama i globalnoj najboljoj poziciji u roju. U kontekstu selekcije instanci, svaka čestica reprezentuje kandidat podskup, kodiran kao binarna sekvenca slično kao kod GA, ali sa dodatnom informacijom o brzini (velocity) koja određuje kako se pozicija čestice menja tokom vremena.

PSO tipično postiže sličnu redukciju kao GA (70-85%) ali sa potencijalno bržom konvergencijom jer je informacija deljenja kroz global best efikasnija od genetskih operatora. Vremenska složenost je $O(I \cdot P \cdot n \cdot T_{eval})$ što je slično GA ali sa manjom konstantom jer PSO obično koristi manje čestica od GA i konvergira brže.

U praksi, evolucione metode se najčešće koriste kada je potrebna visoka tačnost i kada je vreme izvršavanja manje kritično, kao što su offline pripreme trening skupova za kritične aplikacije ili istraživačke studije gde se traži najbolje moguće rešenje bez obzira na trošak. Za rutinske primene sa ograničenjima vremena, determinističke metode kao što je DROP3 su preferirana opcija jer daju dobra rešenja u predvidljivom vremenu.

4. EVALUACIJA

Primena selekcije instanci u realnim sistemima zahteva razmatranje više faktora koji prevazilaze samo teorijske performanse algoritama na benchmark skupovima podataka. Pored tačnosti i redukcije, neophodno je uzeti u obzir vreme izvršavanja, memorijske zahteve, robusnost na promene distribucije, i interpretabilnost rezultata.

4.1 Metrike evaluacije

Evaluacija metoda selekcije instanci zahteva više-dimenzionalni pristup koji kvantifikuje različite aspekte performansi.

Metrike kvaliteta klasifikacije obuhvataju nekoliko povezanih mera. Apsolutna tačnost (accuracy) predstavlja osnovnu meru: $Acc(S) = \text{broj_pravilno_klasifikovanih} / \text{ukupan_broj_instanci}$. Međutim, relevantnija je relativna tačnost koja poredi sa originalnim skupom: $Retention(S) = Acc(S) / Acc(T) \cdot 100\%$, gde vrednost blizu 100% ukazuje na održan kvalitet, $>100\%$ na poboljšanje zbog uklanjanja šuma, a značajno $<100\%$ na gubitak kritičnih informacija.

Za nebalansirane skupove koriste se precision, recall i F1-score metrike. Precision meri koliko od instanci klasifikovanih kao pozitivne su zaista pozitivne: $Precision = TP / (TP + FP)$. Recall meri koliko od stvarnih pozitivnih instanci je pravilno klasifikovano: $Recall = TP / (TP + FN)$. F1-score predstavlja harmonijsku sredinu: $F1 = 2 \cdot (Precision \cdot Recall) / (Precision + Recall)$.

Metrike efikasnosti redukcije kvantifikuju smanjenje veličine podataka. Reduction Rate: $RR = (1 - |S|/|T|) \cdot 100\%$, gde vrednost od 80% znači da je 80% instanci uklonjeno.

Speedup Factor meri ubrzanje klasifikacije: $\text{Speedup} = \text{Time_classify}(T) / \text{Time_classify}(S)$. Za k-NN, ako je S 10% veličine T, teoretski speedup je oko $10\times$.

Metrike računske efikasnosti mere trošak procesa selekcije.

- Selection Time meri koliko dugo algoritam traje, što varira od sekundi za ENN do sati ili dana za GA.
- Memory Usage predstavlja memorijske zahteve tokom selekcije, što može biti kritično za algoritme koji skladište matrice rastojanja $O(n^2)$.
- Scalability predstavlja kako performanse rastu sa veličinom podataka.

4.2 Oblasti primene

Medicinska dijagnostika predstavlja jedan od najzahtevnijih domena zbog kritičnosti tačnosti. Greške mogu imati ozbiljne posledice za zdravlje pacijenata što znači da tačnost ima apsolutni prioritet. Medicinski skupovi često sadrže šum zbog varijabilnosti dijagnostičkih kriterijuma i nesavršenih instrumenata. Edition metode kao ENN ili DROP3 sa konzervativnim parametrima (veće k, niži prag) pokazuju se efikasnim. Retke bolesti dovode do ekstremno nebalansiranih skupova što zahteva pažljivu primenu selekcije koja neće dodatno smanjiti već mali broj pozitivnih primera.

Detekcija prevara u finansijskim transakcijama zahteva klasifikaciju (milisekunde) u realnom vremenu sa ekstremnom nebalansiranošću (0.1-0.5% prevara). Tipičan pristup je separata selekcija po klasama, agresivna kondenzacija većinske klase (redukcija 95%+) uz konzervativnu edition manjinsku klasu (10-20%). Dodatno, sistemi za detekciju prevara rade sa promenom distribucije podataka (concept drift) jer se obrasci prevara menjaju tokom vremena, što zahteva inkrementalne metode selekcije.

Text classification uključuje klasifikaciju dokumenata (spam, sentiment, topics) sa velikim (milioni) i visokodimenzionalnim (desetine hiljada reči) skupovima koji su sparse. CNN i prototype selection metode su efikasne, ali zahtevaju prilagođene metrike rastojanja (cosine similarity umesto Euclidean). Spam detection ima problema sa primerima osmišljenim da obmanu model, što zahteva robusne metode.

Computer vision primenjuje selekciju na naučene reprezentacije iz dubokih mreža (512-dim feature vektori iz CNN-a) umesto na sirovim pikselima. Dataset distillation pokušava da sintetizuje mali set reprezentativnih primera koji sadrže istu informaciju kao veliki skup.

IoT i sensor data uključuju kontinuirane tokove podataka sa milionima senzora gde volumen toliko veliki da skladištenje svih merenja nije praktično. Online/incrementalne verzije selekcije su kritične jer podaci pristižu kontinuirano. Concept drift je čest jer se uslovi rada menjaju.

4.3 Praktične preporuke

Metode kondenzacije (CNN, RNN) su odgovarajuće kada je primarni cilj drastična redukcija (70-95%), podaci su relativno čisti, klasteri kompaktni sa jasnim

granicama, i ciljna aplikacija je k-NN. CNN je brži izbor za prototipove, dok RNN zahteva prohibitivno vreme ($O(n^4)$). Ograničenja uključuju zadržavanje šumnih instanci na granicama i osetljivost na redosled.

Metode uređivanja (ENN, RENN, All k-NN) su najbolji izbor kada kvalitet podataka ima prioritet, sumnja se na prisustvo šuma ($>5\%$), originalni skup je već relativno mali, i cilj je poboljšati generalizaciju. ENN sa $k=3$ je podrazumevani izbor, RENN se koristi kada se sumnja na klastere šuma, a All k-NN je najkonzervativnija opcija za kritične aplikacije.

Hibridne metode (DROP3, ICF) predstavljaju najpouzdaniji izbor za opštu primenu jer konsistentno daju dobre rezultate (80-90% redukcija uz održanu ili poboljšanu tačnost) na širokom spektru skupova podataka. DROP3 je postao opšteprihvaćen u praksi. ICF može biti brža alternativa za velike skupove ali zahteva pažljivo podešavanje threshold parametara.

Evolucione metode (GA, PSO) su opravdane kada je potrebna maksimalna tačnost bez obzira na vreme, optimizacija je za specifičan klasifikator sa složenom fitness funkcijom, postoje multiple konfliktne ciljeve (multi-objective), ili u istraživačkom kontekstu. Zahtevaju expertise u podešavanju parametara i dugotrajno izvršavanje (sati-dani). Praktična preporuka je koristiti evolucione metode za offline pripremu finalnog production modela.

Strategije za nebalansirane skupove: kada je jedna klasa mnogo brojnija, primenjuje se separata selekcija po klasama. Agresivna kondenzacija veće klase, konzervativna edition manje klase, ili cost-sensitive fitness funkcije koje penalizuju uklanjanje manjinske instance.

Strategije za visokodimenzionalne podatke: kada je $d > 1000$, curse of dimensionality čini distance metrics manje diskriminativnim. Preporuka je prvo primeniti feature selection ili PCA za redukciju na 20-100 dimenzija, a zatim instance selection. Alternativno, metrike prilagođene visokim dimenzijama kao što je cosine similarity ili Mahalanobis distance mogu poboljšati rezultate.

U aplikacijama gde novi podaci kontinuirano pristižu, skupne metode koje zahtevaju ponovnu obradu celokupnog skupa nisu praktične. Inkrementalne varijante ENN ili CNN gde se novi primeri procenjuju pojedinačno i dodaju ili odbacuju na osnovu konzistentnosti sa trenutnim skupom mogu održavati kvalitet selekcije. Pristup klizećeg prozora (Sliding Window) periodično uklanja stare instance i dodaje nove, čime se adresira promena distribucije podataka.

Izbor optimalne metode i parametara zahteva rigoroznu evaluaciju kroz stratifikovanu k-struku unakrsnu validaciju (tipično $k=5$ ili 10) gde se selekcija primenjuje samo na skupove za treniranje a evaluacija na skup za validaciju. Za male uzorke ispod 100 instanci, unakrsna validacija izostavljanjem jedne instance može biti neophodna. Dodatno, treba evaluirati stabilnost rezultata kroz višestruka pokretanja sa različitim

početnim vrednostima generatora slučajnih brojeva, posebno za metode osjetljive na randomizaciju kao što su CNN i GA.

5. ZAKLJUČAK

Selekcija instanci predstavlja ključnu tehniku u mašinskom učenju koja omogućava značajnu optimizaciju instance-based algoritama kroz inteligentno smanjenje veličine trening skupova uz održavanje ili poboljšanje kvaliteta modela. Ovaj rad je pružio sveobuhvatan pregled problema selekcije instanci, teoretskih osnova, algoritmičkih pristupa i praktičnih smernica za primenu u realnim sistemima.

Kroz analizu formalnog problema utvrđeno je da selekcija instanci predstavlja multi-objective optimizacioni zadatak sa inherentno konfliktnim ciljevima, maksimizacijom redukcije i maksimizacijom tačnosti. Ovaj fundamentalni trade-off motiviše razvoj različitih kategorija metoda sa različitim prioritetima. Metode kondenzacije fokusiraju se na dramatičnu redukciju (70-95%) zadržavanjem boundary instanci, ali sa rizikom zadržavanja šumnih instanci. Metode uređivanja prioritizuju kvalitet kroz uklanjanje šuma (10-30% redukcija) sa potencijalnim poboljšanjem tačnosti za 2-5%. Hibridne metode, posebno DROP3, kombinuju oba pristupa postižući optimalan balans: redukciju od 80-90% uz održanu ili poboljšanu tačnost.

Analiza algoritama pokazala je da ne postoji univerzalno najbolja metoda, već da izbor zavisi od karakteristika podataka i zahteva aplikacije. CNN pruža brzu i agresivnu kondenzaciju pogodnu za sisteme u realnom vremenu, ali je osjetljiv na redosled i šum. ENN efikasno čisti podatke i često poboljšava generalizaciju, ali postiže umerenu redukciju. DROP3 se pokazao kao najrobusniji izbor za opštu primenu, konsistentno postižući superiorni balans na širokom spektru problema. Evolucione metode mogu naći bliska optimalna rešenja ali zahtevaju prohibitivno vreme izvršavanja.

Praktične smernice razvijene u radu naglašavaju važnost kontekstualne primene. U medicinskim aplikacijama gde je tačnost kritična, edition metode sa konzervativnim parametrima su preferabilne. U finansijskoj detekciji prevara sa ekstremnom nebalansiranošću, separata selekcija po klasama daje najbolje rezultate. Za visokodimenzionalne podatke neophodna je redukcija atributa pre primene instance selection. Streaming aplikacije zahtevaju inkrementalne metode koje mogu ažurirati selektovani skup bez reprocessiranja svih podataka.

Značaj selekcije instanci u savremenim sistemima mašinskog učenja ne može se preceniti. U eri velikih podataka gde skupovi sa milionima instanci postaju norma, inteligentna redukcija ne samo što omogućava praktičnu primenu instance-based algoritama već često dovodi do boljih performansi kroz uklanjanje šuma i redundancije. Rezultati pokazuju da kvalitet podataka često ima veći uticaj na performanse modela nego sam izbor algoritma.

Budući pravci istraživanja uključuju razvoj online i inkrementalnih metoda za streaming podatke, integraciju sa deep learning pristupima gde se selekcija primenjuje na naučene

reprezentacije, multi-objective optimizacione metode koje eksplicitno balansiraju više kriterijuma, adaptive metode koje automatski biraju strategiju na osnovu karakteristika podataka, i proširenje na nove domene kao što su few-shot learning i federated learning. Selekcija instanci će nastaviti da bude relevantna i važna tehnika, ne kao univerzalno rešenje već kao vredan alat u arsenalu praktičara koji razumeju njene prednosti, ograničenja i optimalne kontekste primene.

6. PRAKTIČAN DEO

U praktičnom delu rada primenjene su odabrane metode selekcije instanci koje su najrelevantnije za korišćen k-NN klasifikator. Konkretno, primenjene su edition metode (ENN, RENN, AllKNN), metode kondenzacije (CNN, Tomek Links) i hibridna metoda DROP3, dok su ostale metode poput RNN, MSS, ICF i evolucionih pristupa opisane u teorijskom delu rada radi potpunijeg pregleda tehnika selekcije instanci. Svi eksperimenti su realizovani u Python programskom jeziku korišćenjem biblioteka pandas, scikit-learn, imbalanced-learn i matplotlib, a implementacija je dostupna kroz Jupyter notebook na javnom git repozitorijumu.

6.1 Opis skupa podataka

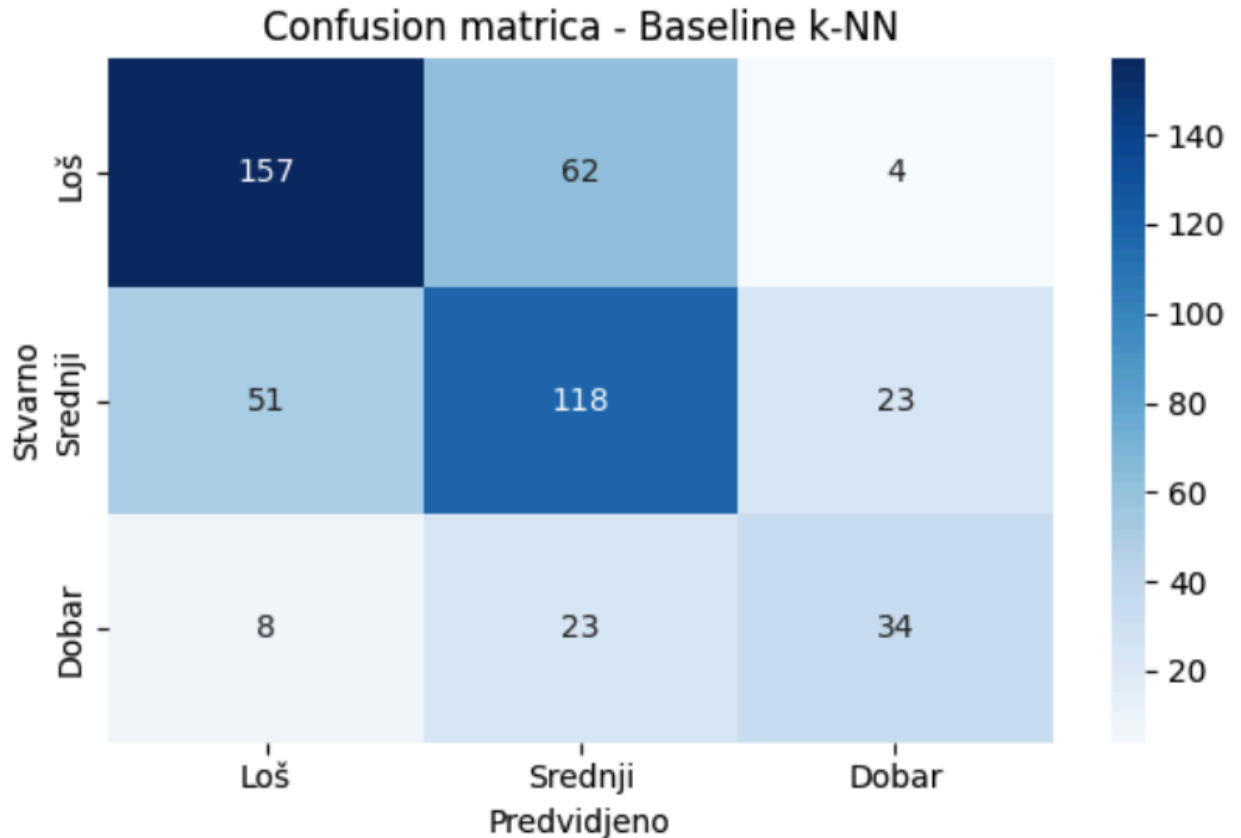
Za praktičnu demonstraciju metoda opisanih u prethodnim poglavljima korišćen je Wine Quality (Red) dataset sa Kaggle platforme. (<https://www.kaggle.com/datasets/uciml/red-wine-quality-cortez-et-al-2009>) Radi se o skupu od 1599 instanci crvenog vina, gde je svaka instanca opisana sa 11 fizičko-hemijskih atributa kao što su kiselost, sadržaj šećera, hloridi, pH vrednost, procenat alkohola i slično. Ciljna promenljiva je ocena kvaliteta vina na skali od 3 do 8, dodeljena od strane senzornih ocenjivača.

Problem sa originalnim oznakama je što pojedine klase imaju izuzetno mali broj primeraka. Klase 3 i 8 sadrže svega 10 i 18 instanci, što je nedovoljno za pouzdanu klasifikaciju i evaluaciju. Zbog toga su ocene grupisane u tri kategorije: loš kvalitet (ocene 3-5) sa 744 instance, srednji kvalitet (ocena 6) sa 638 instanci i dobar kvalitet (ocene 7-8) sa 217 instanci. Ni nakon grupisanja klase nisu balansirane. Klasa „dobar” ima skoro 3.5 puta manje primeraka od klase „loš”, ali svaka klasa sada ima dovoljno instanci za smislenu evaluaciju.

Kao klasifikator izabran je k-Nearest Neighbors (k-NN) sa $k=5$. Ovaj izbor nije slučajna, k-NN je algoritam koji čuva ceo trening skup u memoriji i koristi ga direktno pri klasifikaciji, pa svaka promena u broju ili sastavu trening instanci ima neposredan uticaj na vreme i kvalitet predikcije. To ga čini prirodnim izborom za demonstraciju efekata selekcije instanci. Podaci su podeljeni na trening skup (70%, 1119 instanci) i test skup (30%, 480 instanci) uz stratifikaciju, a numerički atributi su standardizovani pre primene algoritama.

6.2 Baseline i primenjene metode

Pre primene bilo koje metode selekcije, uspostavljen je baseline. Algoritam k-NN je treniran na kompletnom trening skupu, bez ikakvih modifikacija. Baseline tačnost iznosi 0.6438 uz weighted F1-score od 0.6444. Relativno niska polazna tačnost ukazuje na značajno preklapanje između klasa „loš” i „srednji” u prostoru atributa, što se potvrđuje i analizom confusion matrice gde su najčešće greške zamene upravo između ove dve klase.

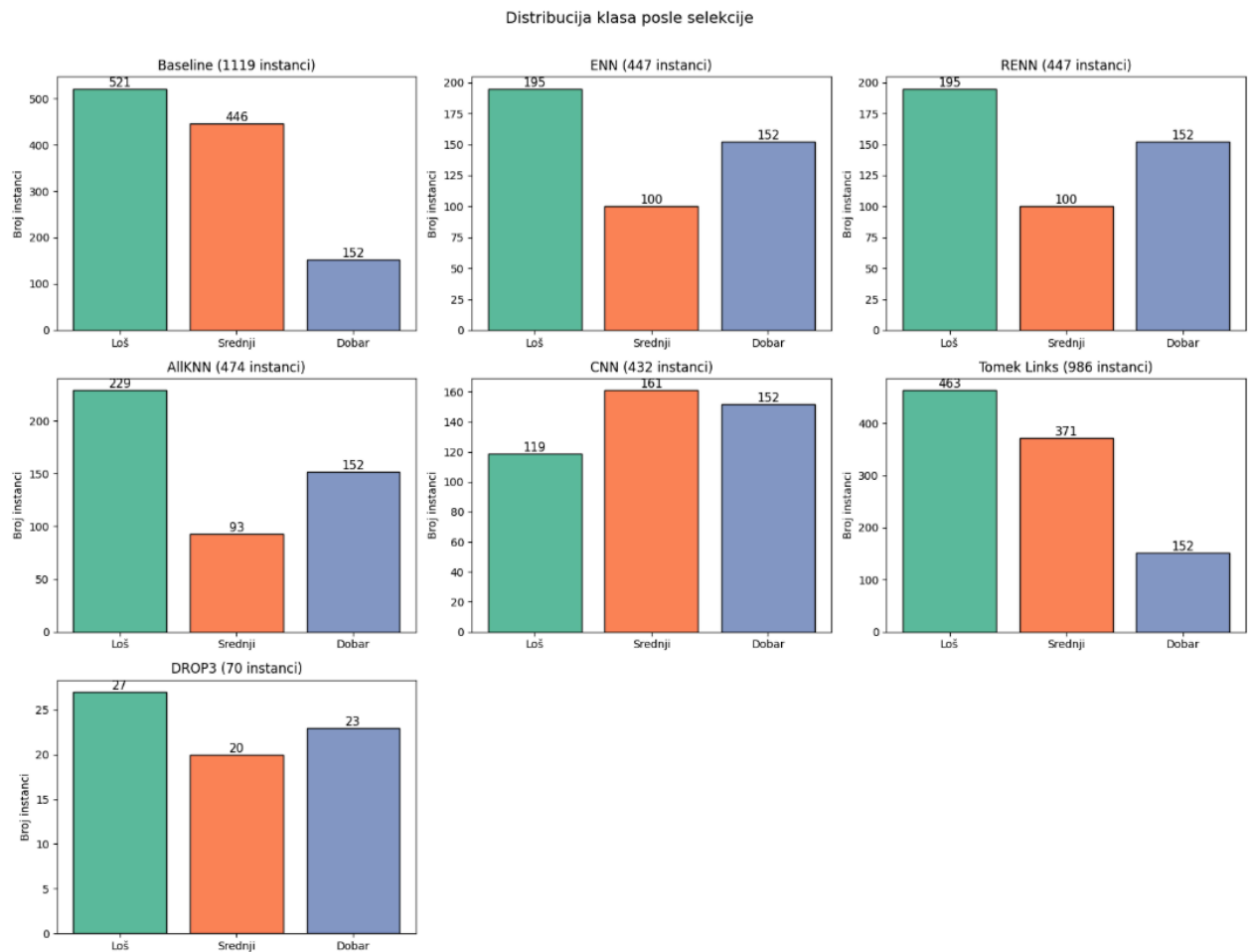


Slika 1: Prikaz matrice konfuzije za baseline k-NN

Na trening skup primenjeno je šest metoda iz tri kategorije obrađene u teorijskom delu:

- Edition metode: ENN, RENN i AllKNN (korišćenjem biblioteke imbalanced-learn)
- Kondenzacija: CNN i Tomek Links (takođe iz imbalanced-learn)
- Hibridna metoda: DROP3 (ručna implementacija prema originalnom radu Wilson & Martinez, 2000, jer ne postoji gotova implementacija u standardnim Python bibliotekama)

Za svaku metodu praćeni su isti parametri: broj preostalih instanci, stopa redukcije, tačnost na test skupu, F1-score i vreme predikcije.



Slika 2: Prikaz distribucije klasa pre i posle selekcije

6.3 Rezultati

Uporedni rezultati prikazani su u tabeli i na Slici 3.

Metoda	Instanci	Redukcija	Tačnost	F1 (weighted)	Vreme predikcije
Baseline	1119	0%	0.6438	0.6444	0.0062
ENN	447	60.1%	0.5583	0.5560	0.0026
RENN	447	60.1%	0.5583	0.5560	0.0028
AIKNN	474	57.6%	0.5750	0.5723	0.0026
CNN	432	61.4%	0.5500	0.5516	0.0025
Tomek Links	986	11.9%	0.6375	0.6382	0.0055
DROP3	70	93.7%	0.5062	0.5150	0.0010



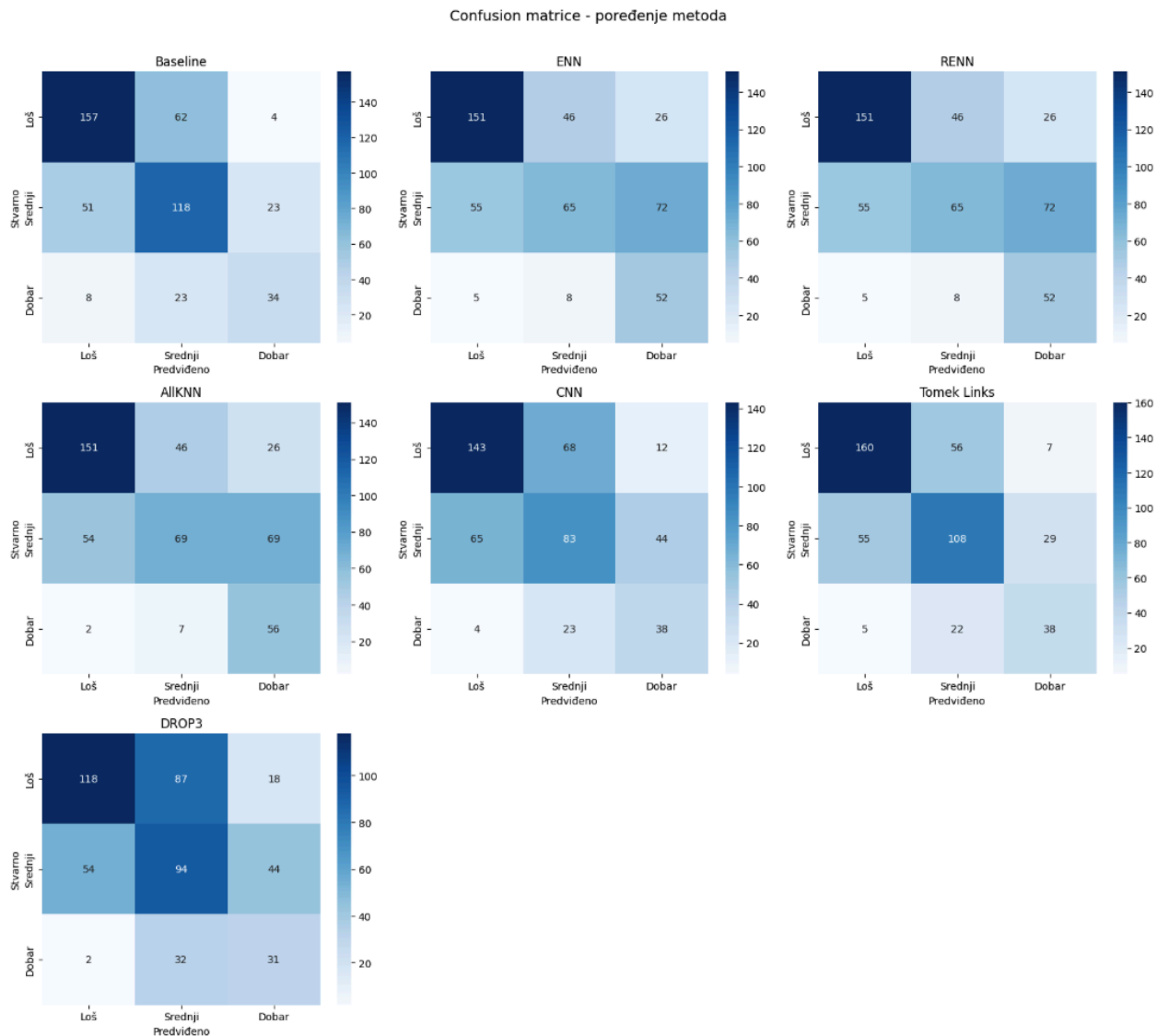
Slika 3: Prikaz rezultata različitih metoda selekcije instanci

Edition metode (ENN, RENN, AIKNN) uklonile su 57-60% instanci, ali su pritom srušile tačnost za 7-9 procentnih poena. RENN je dao identičan rezultat kao ENN jer je konvergirao u jednoj iteraciji. Svi detektibilni šumni klasteri uklonjeni su već u prvom prolazu. AIKNN je bio nešto konzervativniji i sačuvao 27 instanci više, uz blago bolju tačnost.

CNN kao metoda kondenzacije postigao je redukciju od 61.4% ali i najnižu tačnost među ne-hibridnim metodama (0.55), jer zadržava granične instance bez razlikovanja validnih od šumnih.

DROP3 je postigao najagresivniju redukciju (93.7%), svodeći trening skup na svega 70 instanci, ali uz tačnost od samo 0.506. Nakon ENN faze koja je ostavila 447 instanci, dekrementalna kondenzacija dodatno je uklonila sve instance čije pridružene instance (associates) ostaju pravilno klasifikovani.

Tomek Links se pokazao kao najstabilnija metoda, uz najmanju redukciju (11.9%) sačuvao je tačnost praktično identičnu baseline-u (0.6375 vs 0.6438), što potvrđuje da konzervativniji pristup bolje čuva kvalitet klasifikacije kada su klase jako isprepletane.



Slika 4: Prikaz matrice konfuzije radi poređenja metoda

6.4 Diskusija

Na ovom datasetu nijedna metoda selekcije instanci nije poboljšala tačnost u odnosu na polazni model. To ne znači da su metode neuspešne, znači da su karakteristike dataseta takve da granice između klasa nisu jasno definisane, pa svako uklanjanje instanci gubi informacije umesto da ih prečisti. Ovo je u kontrastu sa tipičnim rezultatima iz literature gde DROP3 postiže 80-90% redukcije uz održanu ili poboljšanu tačnost, ali potpuno u skladu sa očekivanjima za skup sa ovakvim stepenom preklapanja klasa.

ENN je uklonio 60.1% instanci, što je znatno više od tipičnih 10-30% navedenih u literaturi, što direktno ukazuje na izraženo preklapanje klasa u ovom skupu. CNN je postigao redukciju od 61.4%, ispod tipičnih 70-95% iz literature, iz istog razloga, jer na skupovima sa razuđenim granicama između klasa CNN mora da zadrži više graničnih instanci. Za razliku od tipičnih rezultata opisanih u teorijskom delu, DROP3 nije sačuvao tačnost već je zabeležio najveći pad, jer se u procesu kondenzacije gube instance koje nose informaciju o nedovoljno jasnim granicama između klasa.

Rezultati ipak jasno ilustruju razlike između kategorija metoda opisanih u teorijskom delu rada i potvrđuju da ne postoji univerzalno najbolja metoda. Izbor zavisi od karakteristika konkretnog skupa podataka. Na datasetima sa kompaktnim klasterima i jasnim granicama, agresivne metode poput CNN ili DROP3 mogu postići veliku redukciju bez pada tačnosti. Na datasetima sa izraženim preklapanjem, kakav je ovaj, konzervativni pristupi poput Tomek Links-a su bezbedniji izbor.

Kompletna implementacija sa svim vizualizacijama dostupna je kao Jupyter notebook na javnom git repozitorijumu:

https://github.com/alxcek/mas-ai/tree/master/data_preprocessing/seminar_paper_1.

7. LITERATURA

- [1] Hart, P. E. (1968). The condensed nearest neighbor rule. *IEEE Transactions on Information Theory*, 14(3), 515-516.
- [2] Wilson, D. L. (1972). Asymptotic properties of nearest neighbor rules using edited data. *IEEE Transactions on Systems, Man, and Cybernetics*, 2(3), 408-421.
- [3] Gates, G. W. (1972). The reduced nearest neighbor rule. *IEEE Transactions on Information Theory*, 18(3), 431-433.
- [4] Brighton, H., & Mellish, C. (1999). On the consistency of information filters for lazy learning algorithms. In *Proceedings of the Third European Conference on Principles of Data Mining and Knowledge Discovery* (pp. 283-288). Springer.
- [5] Brighton, H., & Mellish, C. (2002). Advances in instance selection for instance-based learning algorithms. *Data Mining and Knowledge Discovery*, 6(2), 153-172.
- [6] Wilson, D. R., & Martinez, T. R. (2000). Reduction techniques for instance-based learning algorithms. *Machine Learning*, 38(3), 257-286.
- [7] García, S., Derrac, J., Cano, J., & Herrera, F. (2012). Prototype selection for nearest neighbor classification: Taxonomy and empirical study. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 34(3), 417-435.
- [8] Olvera-López, J. A., Carrasco-Ochoa, J. A., Martínez-Trinidad, J. F., & Kittler, J. (2010). A review of instance selection methods. *Artificial Intelligence Review*, 34(2), 133-153.
- [9] Cano, J. R., Herrera, F., & Lozano, M. (2003). Using evolutionary algorithms as instance selection for data reduction in KDD: An experimental study. *IEEE Transactions on Evolutionary Computation*, 7(6), 561-575.
- [10] Derrac, J., García, S., & Herrera, F. (2010). IFS-CoCo: Instance and feature selection based on cooperative coevolution with nearest neighbor rule. *Pattern Recognition*, 43(6), 2082-2105.
- [11] Liu, H., & Motoda, H. (2001). *Instance Selection and Construction for Data Mining*. Springer Science & Business Media.

- [12] Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21-27.
- [13] Aha, D. W., Kibler, D., & Albert, M. K. (1991). Instance-based learning algorithms. *Machine Learning*, 6(1), 37-66.
- [14] Devijver, P. A., & Kittler, J. (1980). On the edited nearest neighbor rule. In *Proceedings of the Fifth International Conference on Pattern Recognition* (pp. 72-80). IEEE.
- [15] Tomek, I. (1976). Two modifications of CNN. *IEEE Transactions on Systems, Man, and Cybernetics*, 6(11), 769-772.
- [16] Herbst, S. (2020). An Introduction to Instance Selection in Data Mining. *Medium*. Preuzeto sa <https://medium.com/@sabrinaherbst/an-introduction-to-instance-selection-in-data-mining-7b2ac94fb07>
- [17] Instance selection. (2024). U *Wikipedia*. Preuzeto sa https://en.wikipedia.org/wiki/Instance_selection
- [18] Dua, D., & Graff, C. (2017). UCI Machine Learning Repository. University of California, Irvine, School of Information and Computer Sciences. Preuzeto sa <http://archive.ics.uci.edu/ml>
- [19] García, S., & Herrera, F. (2008). An extension on "statistical comparisons of classifiers over multiple data sets" for all pairwise comparisons. *Journal of Machine Learning Research*, 9, 2677-2694.
- [20] Alcalá-Fdez, J., Fernández, A., Luengo, J., Derrac, J., García, S., Sánchez, L., & Herrera, F. (2011). KEEL data-mining software tool: Data set repository, integration of algorithms and experimental analysis framework. *Journal of Multiple-Valued Logic & Soft Computing*, 17, 255-287.